



Санкт-Петербургский государственный университет

Статический полиморфизм в C++

Хромов Иван Алексеевич, Козенко Дмитрий Сергеевич, Пучкин Станислав Свагович,
группа 23.Б08-мм

Санкт-Петербург
2025

Полиморфизм — способность ассоциировать различные конкретные поведения с единой обобщённой нотацией.

Виды полиморфизма в C++

- **Run-time** (интерфейсы, `std::any`, использование мутабельных указателей на функции)
- **Link-time** (weak-символы, динамическая линковка)
- **Compile-time** (?)

Перегрузка функций

Механизм перегрузки функций

- Находим все функции, отвечающие заданному имени (name lookup)
- Выбираем наиболее подходящую (best viable function) из найденных

Перегрузка функций

```
void print(int value) {  
    std::cout << "int: " << value << "\n";  
}  
  
void print(double value) {  
    std::cout << "double: " << value << "\n";  
}  
  
void print(const std::string& value) {  
    std::cout << "string: " << value << "\n";  
}  
  
int main() {  
    print(42);                // print(int)  
    print(3.14);              // print(double)  
    print(std::string("hello")); // print(const string&)  
}
```

Специализация шаблонов

Специализация функций

```
template <class T>
void Foo(T) {
    std::cout << "1" << '\n';
}

template <>
void Foo<int>(int) {
    std::cout << "2" << '\n';
}

int main () {
    Foo("foo"); // "1"
    Foo(42);    // "2"
}
```


Частичная специализация класса

```
template<typename T>
struct Storage {
    static void show() { std::cout << "Обычный тип\n"; }
};

// Частичная специализация для указателей
template<typename T>
struct Storage<T*> {
    static void show() { std::cout << "Указатель на " << typeid(T).name() << '\n'; }
};

int main() {
    Storage<int>::show();    // Обычный тип
    Storage<int*>::show();  // Указатель на int
}
```

Полная специализация класса

```
template<typename T>
class Printer {
public:
    static void print(T value) {
        std::cout << "Generic: " << value << "\n";
    }
};

template<>
class Printer<int> {
public:
    static void print(int value) {
        std::cout << "Int: " << value * 2 << "\n";
    }
};

int main() {
    Printer<int>::print(5);           // Int: 10
    Printer<double>::print(3.14);    // Generic: 3.14
}
```

if constexpr

if constexpr

```
template<typename T>
void process(T value) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "Целочисленный тип: " << value * 2 << '\n';
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "Число с плавающей точкой: " << value / 2 << '\n';
    } else {
        std::cout << "Другой тип: " << value << '\n';
    }
}

int main() {
    process(10);      // Целочисленный тип: 20
    process(3.14);    // Число с плавающей точкой: 1.57
    process("csgs forever"); // Другой тип: csgs forever
}
```

Concepts and requires

requires

```
template <class V> requires (sizeof(V) < 32)
void wrap(V) {
    std::cout << "small type\n";
}

template <class V> requires (sizeof(V) >= 32)
void wrap(V) {
    std::cout << "large type\n";
}

int main() {
    wrap(char{});           // small type
    wrap(double{});        // small type
    struct Big { char data[32]; };
    wrap(Big{});           // large type (32 >= 32)
}
```

```
template <class V> requires (sizeof(V) < 32)
class Wrapper {};

template <class V> requires (sizeof(V) >= 32)
class Wrapper {};

// error: template class redefinition with different requires clause
```

Requires expressions

Requires expressions

```
template <class It>
int distance(It l, It r) {
    if constexpr (requires { r - l; }) {
        return r - l;
    } else {
        // loop here
    }
}
```

Concepts

```
template<typename T>
concept Integral = std::is_integral_v<T>;

void print(Integral auto value) {
    std::cout << "Integer: " << value << std::endl;
}

int main() {
    print(42);    // Integer: 42
    print(3.14); // error: double does not satisfy Integral
}
```

```
template<typename T>
concept Small = sizeof(T) < 32;

template<typename T>
concept Large = sizeof(T) >= 32;

template<Small T>
void wrap(T) { std::cout << "small\n"; }

template<Large T>
void wrap(T) { std::cout << "large\n"; }
```

C RTP — Curiously Recurring Template
Pattern

```
template<typename Derived>
class Shape {
public:
    double area() const {
        return static_cast<const Derived*>(this)->areaImpl();
    }
};

class Circle : public Shape<Circle> {
    double r_;
public:
    Circle(double r) : r_(r) {}
    double areaImpl() const { return 3.14159 * r_ * r_; }
};

class Rectangle : public Shape<Rectangle> {
    double w_, h_;
public:
    Rectangle(double w, double h) : w_(w), h_(h) {}
    double areaImpl() const { return w_ * h_; }
};
```